

Design Patterns

Contents

What patterns are for

1. [What a design pattern is](#)
2. [The three categories](#)
3. [Patterns and principles](#)

Creational

4. [Singleton](#)
5. [Factory Method](#)
6. [Abstract Factory](#)
7. [Builder](#)

Structural

8. [Adapter](#)
9. [Decorator](#)
10. [Facade](#)
11. [Proxy](#)
12. [Composite](#)

Behavioral

13. [Strategy](#)
14. [State](#)
15. [Observer](#)
16. [Template Method](#)
17. [Command](#)
18. [Chain of Responsibility](#)

Where these already appear

19. [Patterns in the JDK](#)
20. [Patterns in Spring](#)

Worked example

21. [Replacing a switch with Strategy](#)

Reference

- 22. [Interview questions](#)
- 23. [Common mistakes](#)
- 24. [Quick reference](#)

1. What a design pattern is

A design pattern is a named, reusable solution to a problem that keeps appearing in object-oriented design. It is not a library and not code we copy. It is a shape, described well enough that two engineers can say the name and picture the same arrangement of classes.

The catalogue everyone means comes from the 1994 book *Design Patterns*, by Erich Gamma, Richard Helm, Ralph Johnson and John Vlissides. They are usually called the **GoF**, the **Gang of Four**, and the book describes 23 patterns.

Two things are worth carrying into an interview.

The main value is the vocabulary. Saying "we put the fee rules behind a Strategy" tells a colleague the whole design in six words. Without the name, that is three minutes of explanation and a whiteboard.

Patterns are answers, so the question has to exist first. Each one solves a specific problem and charges a price in indirection. A pattern should be the thing we arrive at, not the thing we start from.

2. The three categories

The Gang of Four sorted the 23 patterns by what each one is about.

Category	Concern	Patterns
Creational	How objects get made	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	How objects are put together	Adapter, Decorator, Facade, Proxy, Composite, Bridge, Flyweight
Behavioral	How objects talk and share work	Strategy, State, Observer, Template Method, Command, Chain of Responsibility, Iterator, Mediator, Memento, Visitor, Interpreter

Being able to place a pattern in its category is a common opening question, and it is easy marks.

This document covers the fifteen that actually appear in interviews and in ordinary Java work. The rest are worth recognizing by name: **Prototype** copies an existing object instead of building a new one, **Bridge** separates an abstraction from its implementation so both can vary, **Flyweight** shares one instance across many uses to save memory, **Mediator** puts the rules for how a group of objects interact into one place, **Memento**

captures state so it can be restored later, **Visitor** adds an operation to a set of classes without editing them, and **Interpreter** evaluates a small language. **Iterator** is so standard that Java built it into the language with `Iterable` and the for-each loop.

3. Patterns and principles

The previous document in this set covers design principles. The relationship between the two is worth being able to state, because interviewers ask questions that sit across both.

A principle says which direction is better. A pattern is a known route in that direction. A principle can be satisfied with no named pattern at all, and a pattern applied without the principle behind it is decoration.

Principle	What it asks for	A pattern that delivers it
Open closed	Add behavior without editing working code	Strategy, Decorator
Dependency inversion	Depend on an abstraction we define ourselves	Adapter
Composition over inheritance	Hold behavior rather than inherit it	Decorator, Strategy
Separation of concerns	One kind of work in one place	Facade
Single responsibility	One reason to change	Command
Low coupling	The caller knows as little as possible	Observer, Facade
Information hiding	Hide the decision most likely to change	Facade, Proxy
Fail fast	Reject bad state at the earliest point	Builder, validating in <code>build</code>
Tell, Don't Ask	Behavior lives with the data it works on	State

The table reads in one direction only. Adding a Strategy does not by itself make a design open closed, and a switch statement that never changes already satisfies the principle without any pattern at all.

4. Singleton

Intent. Guarantee a class has one instance within a defined scope, and give callers a single point of access to it.

The scope matters, and stating it precisely is worth a mark. The Gang of Four wording is "one instance" with "a global point of access", but in Java the guarantee is **per class loader, not per JVM**. Two class loaders loading the same class produce two instances, which is why application servers, OSGi containers and plugin systems surprise people. In Spring the scope is the application context, so two contexts hold two beans.

When the singleton fits the enum model, a single-element enum is the form to prefer, and Joshua Bloch argues for it in *Effective Java*:

```
public enum Clock {
    INSTANCE;

    public Instant now() { return Instant.now(); }
}
```

The language guarantees one instance per class loader, the enum serializes correctly, and it cannot be broken by reflection. No synchronization is needed.

It does not fit every case. An enum already extends `java.lang.Enum`, so it cannot extend anything else, though it can implement interfaces. It is also initialized when the class is first used rather than on our own terms, and some frameworks want to construct the object themselves. Where any of that applies, the holder idiom is the clean alternative, and it is also the form to use when a lazy singleton is genuinely required:

```
public final class Config {
    private Config() { }

    private static final class Holder {
        static final Config INSTANCE = new Config();
    }

    public static Config getInstance() {
        return Holder.INSTANCE;    // Holder loads on first use, once
    }
}
```

The class loader does the locking for us, because a class is initialized once and that initialization is thread safe by specification.

Double-checked locking is worth understanding because it gets asked, even though the two forms above are better:

```
private static volatile Config instance;    // volatile is not optional

public static Config getInstance() {
    if (instance == null) {
        synchronized (Config.class) {
            if (instance == null) {
                instance = new Config();
            }
        }
    }
    return instance;
}
```

Without `volatile` the pattern is broken. The write that publishes the reference can become visible before the constructor has finished, so another thread can read a reference to a partly built object.

The honest answer about Singleton

It is the most overused pattern in the catalogue, though the usual criticism is stated too strongly. A singleton does not have to be mutable, and a stateless one is close to harmless. The real cost is the global access point: a class that calls `Config.getInstance()` does not declare that dependency in its constructor, so the dependency is hidden from every reader and cannot be substituted in a test. When the singleton also holds mutable state, tests additionally become order dependent, because each one inherits whatever the last one left behind. Prefer one instance created at startup and passed in. In Spring that is the normal arrangement, since a bean is one instance per context but is still injected.

5. Factory Method

Intent. Define a method for creating an object, and let subclasses decide which concrete product to create. Also called Virtual Constructor.

```
public interface Notifier {
    void send(String to, String message);
}

public abstract class NotificationService {

    // The factory method. A subclass decides what gets built.
    protected abstract Notifier createNotifier();

    public void notifyUser(String to, String message) {
        createNotifier().send(to, message);
    }
}

public class EmailNotificationService extends NotificationService {
    @Override
    protected Notifier createNotifier() { return new EmailNotifier(); }
}
```

Three things get called "factory" and an interviewer may be checking whether we can separate them:

Name	What it is
Static factory method	A static method returning an instance, instead of a constructor. <code>List.of()</code> , <code>Integer.valueOf()</code> . Not a GoF pattern.
Factory Method	A method a subclass overrides to choose the concrete class, as above.

Name	What it is
Abstract Factory	An object whose methods create a whole family of related products that must match each other. Section 6.

Static factory methods are the one used most often in day-to-day Java, and *Effective Java* opens with them. They have names, so `Money.ofCents(500)` reads better than a constructor; they can return a subtype; and they can reuse instances, which is how `Integer.valueOf` caches the values from -128 to 127.

6. Abstract Factory

Intent. Create families of related objects without naming their concrete classes, and guarantee that the objects in one family are used together.

```
public interface UiFactory {
    Button createButton();
    Checkbox createCheckbox();
}

public class WindowsFactory implements UiFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

public class MacFactory implements UiFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}
```

The caller holds one `UiFactory` and asks it for everything, so a Windows button can never appear beside a Mac checkbox. That guarantee is the whole point, and it is what separates this from a collection of unrelated factory methods.

	Factory Method	Abstract Factory
Creates	One product	A family of related products
Mechanism	A subclass overrides one method	One object with several creation methods
Selected by	Which subclass is in use	Which factory instance is injected
Question it answers	Which concrete class	Which matching set

A real example in the JDK is `java.sql.Connection`. It creates `Statement`, `PreparedStatement` and `CallableStatement`, and every object it hands back belongs to the same driver, which is exactly the family guarantee.

7. Builder

Intent. Build a complex object step by step, so that construction is readable and the finished object can be immutable.

The problem it solves is the telescoping constructor, where four optional fields turn into six overloads and a call site nobody can read:

```
new Order(42, total, null, null, true, false);    // what are those flags?
```

```
public final class Order {

    private final long userId;
    private final BigDecimal total;
    private final String couponCode;
    private final boolean giftWrapped;

    private Order(Builder b) {
        this.userId      = b.userId;
        this.total        = b.total;
        this.couponCode  = b.couponCode;
        this.giftWrapped = b.giftWrapped;
    }

    public static Builder builder() { return new Builder(); }

    public static final class Builder {
        private long userId;
        private BigDecimal total;
        private String couponCode;
        private boolean giftWrapped;

        public Builder userId(long v)          { this.userId = v; return this; }
        public Builder total(BigDecimal v)     { this.total = v; return this; }
        public Builder couponCode(String v)    { this.couponCode = v; return this; }
        public Builder giftWrapped(boolean v)  { this.giftWrapped = v; return this; }

        public Order build() {
            if (total == null || total.signum() <= 0) {
                throw new IllegalArgumentException("total must be positive");
            }
            return new Order(this);
        }
    }
}
```

```
Order order = Order.builder()
    .userId(42)
    .total(new BigDecimal("99.00"))
    .giftWrapped(true)
    .build();
```

Two details an interviewer looks for. The first is where validation goes. Anything involving more than one field belongs in `build`, because that is the first point at which every value is known, and a start date compared against an end date has nowhere else to live. A single setter can still reject a value that is wrong on its own, such as a negative quantity, and failing there gives the caller a better error message. What matters is that no sequence of calls can produce a finished object that is invalid. The second is that every field is `final`, which stops it being reassigned after construction.

final does not mean immutable

A `final` field cannot be reassigned, but the object it points at can still change:

```
private final List<Item> items;
```

is final, and every caller holding that list can still add to it.

The result is immutable, and therefore safe to share between threads, only when every field is either an immutable type or defensively copied on the way in and on the way out. The `Order` above qualifies because `long`, `BigDecimal`, `String` and `boolean` all are. A list field would need `List.copyOf` inside the constructor.

Records cover the simple cases now, and Lombok's `@Builder` generates all of this. The pattern still earns its place when there are many optional fields or when construction needs validation across several of them.

8. Adapter

Intent. Convert the interface a class offers into the interface a caller expects.

This is the pattern that keeps third-party code from leaking through our own design:

```

// What our code wants to depend on. We define it, per dependency inversion.
public interface PaymentGateway {
    Receipt charge(Money amount, String token);
}

// What the vendor SDK actually offers. We do not control this shape.
public class StripeClient {
    public StripeCharge createCharge(long cents, String currency, String source) {
    ... }
}

// The adapter sits between them.
public class StripeGateway implements PaymentGateway {

    private final StripeClient stripe;

    public StripeGateway(StripeClient stripe) { this.stripe = stripe; }

    @Override
    public Receipt charge(Money amount, String token) {
        StripeCharge charge = stripe.createCharge(
            amount.cents(), amount.currency(), token);
        return new Receipt(charge.getId(), amount);
    }
}

```

Every class in the application depends on `PaymentGateway`. Only `StripeGateway` knows Stripe exists, so changing provider means writing one new adapter.

9. Decorator

Intent. Add behavior to an object by wrapping it in another object with the same interface, without touching the original class.

```

public class CachingOrderStore implements OrderStore {

    private final OrderStore delegate;
    private final Map<Long, Order> cache = new ConcurrentHashMap<>();

    public CachingOrderStore(OrderStore delegate) { this.delegate = delegate; }

    @Override
    public Order find(long id) {
        return cache.computeIfAbsent(id, delegate::find);
    }
}

```

Because the wrapper implements the same interface it wraps, decorators stack:

```
OrderStore store = new LoggingOrderStore(  
    new CachingOrderStore(  
        new JpaOrderStore(entityManager)));
```

Each layer adds one behavior and knows nothing about the others. Adding retries later means writing one more decorator, not editing any of these, which is the open closed principle in practice.

`java.io` is the textbook example, and it is worth naming because everyone has typed it:

```
BufferedReader in = new BufferedReader(  
    new InputStreamReader(  
        new FileInputStream(file)));
```

Adapter, Decorator and Proxy, which look identical

All three wrap an object, and interviewers ask the difference often. **Adapter** changes the interface, so the caller can use something it otherwise could not. **Decorator** keeps the interface and adds behavior. **Proxy** keeps the interface and controls access to the real object. Same structure, three different intents, and intent is what names a pattern.

10. Facade

Intent. Put one simple entry point in front of a complicated subsystem.

```
public class CheckoutFacade {  
  
    private final InventoryService inventory;  
    private final PaymentGateway payments;  
    private final ShippingService shipping;  
  
    public OrderConfirmation checkout(Cart cart, PaymentDetails details) {  
        inventory.reserve(cart.items());  
        Receipt receipt = payments.charge(cart.total(), details.token());  
        Shipment shipment = shipping.schedule(cart.address(), cart.items());  
        return new OrderConfirmation(receipt.id(), shipment.trackingNumber());  
    }  
}
```

The caller makes one call instead of orchestrating three services and knowing the order they must run in.

A facade simplifies the common path. It does not seal the subsystem off, and code with an unusual need can still use the services directly. That is the difference between a facade and a badly placed layer that everything has to squeeze through.

11. Proxy

Intent. Stand in for another object, with the same interface, in order to control access to it. Also called Surrogate.

The common kinds: **virtual** proxies delay building an expensive object until it is used, **protection** proxies check permissions first, **remote** proxies represent an object living in another process, and **caching** proxies store results.

Java has direct support through `java.lang.reflect.Proxy`, and this is the mechanism behind a large part of Spring:

```
// Spring wraps the bean in a proxy. The proxy opens the transaction,  
// calls the real method, then commits or rolls back.  
@Transactional  
public void transfer(long from, long to, long cents) { ... }
```

That single fact answers a frequently asked question. Calling `transfer` from another method inside the same class does not go through the proxy, so no transaction starts. The same reasoning explains `@Cacheable`, `@Async` and `@PreAuthorize`, all of which stop working on self-invocation for exactly the same reason, and none of which can apply to a `private` or `final` method.

Hibernate uses proxies too. A lazily loaded association is a proxy, and touching it after the session has closed is what produces `LazyInitializationException`.

12. Composite

Intent. Arrange objects into a tree and let a caller treat a single object and a whole group the same way.

```

public interface Node {
    long sizeInBytes();
}

public record FileNode(String name, long sizeInBytes) implements Node { }

public class FolderNode implements Node {

    private final String name;
    private final List<Node> children = new ArrayList<>();

    public FolderNode(String name) { this.name = name; }

    public void add(Node child) { children.add(child); }

    @Override
    public long sizeInBytes() {
        return children.stream().mapToLong(Node::sizeInBytes).sum();
    }
}

```

The caller asks any `Node` for its size without knowing whether it holds one file or a folder containing a thousand. The recursion lives in the structure rather than in every caller.

The shape appears wherever there are trees: file systems, nested menus, organisation charts, expression trees, and user interface components. The JDK has its own, since `java.awt.Container` extends `Component` and holds a list of `Component`s.

The trade-off worth naming: the shared interface has to make sense for both a leaf and a group. If `add` sits on the common interface, a file has to implement it, and the usual answer is to throw, which is a Liskov substitution violation. Keeping child management on the composite only, as above, avoids that at the cost of the caller sometimes needing the concrete type.

13. Strategy

Intent. Define a family of interchangeable algorithms, put each behind a common interface, and choose between them at run time. Also called Policy.

```

public interface ShippingCost {
    Money costFor(Order order);
}

public class StandardShipping implements ShippingCost {
    public Money costFor(Order order) { return Money.ofCents(500); }
}

public class ExpressShipping implements ShippingCost {
    public Money costFor(Order order) { return Money.ofCents(1500); }
}

```

The caller holds a `ShippingCost` and never knows which one it has. Adding overnight shipping means adding a class, which is the open closed principle again.

In modern Java a strategy with one method is often just a lambda, and the standard library is full of them.

`Comparator` is a strategy for ordering:

```
orders.sort(Comparator.comparing(Order::total).reversed());
```

Section 21 works through replacing a switch statement with this pattern.

14. State

Intent. Let an object change its behavior when its internal state changes, so that it appears to change class.

The problem it solves is the status machine written as a chain of conditionals, repeated in every method that cares:

```

public void advance() {
    if (status == NEW)           { charge();  status = PAID; }
    else if (status == PAID)     { dispatch(); status = SHIPPED; }
    else if (status == SHIPPED)  { confirm();  status = DELIVERED; }
    else throw new IllegalStateException("already delivered");
}

```

Each state becomes a type that knows only its own behavior and which state follows it:

```

public interface OrderState {
    OrderState next(Order order);
}

public class New implements OrderState {
    public OrderState next(Order order) {
        order.charge();
        return new Paid();
    }
}

public class Paid implements OrderState {
    public OrderState next(Order order) {
        order.dispatch();
        return new Shipped();
    }
}

```

Where the transitions live is a choice rather than part of the pattern. Above, each state returns its successor, which keeps the rules for one step in one place. The alternative is to leave the states knowing only their own behavior and let the context decide what follows, which suits a machine whose transitions depend on outside conditions. Both are the State pattern.

Java enums can carry behavior per constant, which is the compact version and the one most often written in practice:

```

public enum OrderStatus {
    NEW      { public OrderStatus next() { return PAID; } },
    PAID     { public OrderStatus next() { return SHIPPED; } },
    SHIPPED  { public OrderStatus next() { return DELIVERED; } },
    DELIVERED { public OrderStatus next() {
        throw new IllegalStateException("terminal state");
    } };

    public abstract OrderStatus next();
}

```

Strategy and State, which are the same shape

Structurally these two are identical: an interface, several implementations, and an object holding one of them. Interviewers ask the difference because the answer shows whether the patterns are understood by intent rather than by diagram.

	Strategy	State
What varies	The algorithm for one task	The behavior of the whole object

	Strategy	State
Who chooses	The caller, usually once at construction	The states themselves, as the object moves
Do the implementations know each other	No, they are independent	Often, when the states own the transition logic
Typical trigger	Configuration, or a request parameter	An event that advances the object
Changes during a lifetime	Rarely	That is the entire point

15. Observer

Intent. When one object changes, everything that depends on it is told automatically.

The Gang of Four book lists **Dependents** and **Publish-Subscribe** as other names for it, but the second one has drifted since 1994 and the difference is worth stating in an interview:

- In **Observer**, the subject holds a list of its observers and calls them directly. It knows who is listening.
- In **publish-subscribe** as the term is normally used now, a broker or event bus sits in between. The publisher names an event, not a listener, and does not know whether anything is subscribed.

That distinction decides real behavior: what happens when a listener fails, whether delivery can be asynchronous, and whether events survive a restart.

`java.util.Observer` and `Observable` exist but were deprecated in Java 9, and saying so is a small signal that we keep up. Modern Java uses listener interfaces, Spring's event publisher, or reactive streams.

```
@Service
public class OrderService {

    private final ApplicationEventPublisher events;

    public long create(Order order) {
        long id = store.create(order);
        events.publishEvent(new OrderCreatedEvent(id, order.email()));
        return id;
    }
}

@Component
public class SendConfirmationEmail {

    @EventListener
    public void on(OrderCreatedEvent event) { ... }
}
```

`OrderService` no longer knows that confirmation emails exist. Adding a second reaction, such as updating a reporting table, means adding a listener and changing nothing here.

Two details worth knowing, because both cause real bugs. Spring events are **synchronous by default**, so a slow listener slows the caller down, and `@Async` changes that. And a listener that runs inside the caller's transaction can see data that is about to be rolled back, which is what `@TransactionalEventListener` and its `AFTER_COMMIT` phase exist for.

The cost of the pattern is traceability. The flow no longer reads top to bottom in one file, and finding what happens after an order is created means searching for listeners.

16. Template Method

Intent. Put the skeleton of an algorithm in a base class and let subclasses fill in particular steps, without changing the order of those steps.

```
public abstract class ImportJob {  
  
    // The template method itself. final, so the sequence cannot be changed.  
    public final ImportResult run(Path file) {  
        List<Row> rows = parse(file);  
        validate(rows);  
        return persist(rows);  
    }  
  
    protected abstract List<Row> parse(Path file);  
  
    protected void validate(List<Row> rows) { }    // optional hook  
  
    protected abstract ImportResult persist(List<Row> rows);  
}
```

Marking `run` as `final` is the detail interviewers look for. The whole point is that the sequence is fixed, and a subclass that can override `run` can break it.

This is the one common pattern built on inheritance, so it carries the fragile base class problem. Strategy is the composition-based alternative, and the trade is worth stating: Template Method keeps the shared sequence in one place, Strategy gives more freedom and can be swapped at run time.

17. Command

Intent. Wrap a request in an object, so it can be passed around, queued, logged, or undone.

```

public interface Command {
    void execute();
    void undo();
}

public class AddItemCommand implements Command {

    private final Cart cart;
    private final Item item;

    public AddItemCommand(Cart cart, Item item) {
        this.cart = cart;
        this.item = item;
    }

    public void execute() { cart.add(item); }
    public void undo()    { cart.remove(item); }
}

```

Keeping a list of executed commands gives undo for free, which is why editors and drawing tools use it.

Java has the shape built in. `Runnable` is a command with one method, and an executor is the queue:

```

executor.submit(() -> orders.create(order));    // a command, queued

```

18. Chain of Responsibility

Intent. Pass a request along a chain of handlers until one of them deals with it, so the sender does not know which handler will respond.

The most familiar version in Java is the servlet filter chain, and Spring Security is a long chain of filters where each one either handles the request, changes it, or passes it on:

```

public class RateLimitFilter implements Filter {

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {

        if (overLimit(req)) {
            ((HttpServletResponse) res).setStatus(429);
            return;                // stop, do not continue the chain
        }
        chain.doFilter(req, res);    // pass it on
    }
}

```

Each handler decides for itself whether to stop or continue, and the order of the chain is configuration rather than code.

19. Patterns in the JDK

Naming these is one of the highest-value things to have ready, because it shows the patterns are recognized rather than memorized.

Pattern	In the JDK
Singleton	<code>Runtime.getRuntime()</code> , <code>Desktop.getDesktop()</code>
Static factory	<code>List.of</code> , <code>Optional.of</code> , <code>Integer.valueOf</code> , <code>Calendar.getInstance()</code> , <code>NumberFormat.getInstance()</code>
Factory Method, similar shape	<code>Collection.iterator()</code> , where each implementation returns its own <code>Iterator</code>
Abstract Factory	<code>Connection</code> creating <code>Statement</code> and <code>PreparedStatement</code>
Builder	<code>StringBuilder</code> , <code>Stream.Builder</code> , <code>HttpRequest.newBuilder()</code>
Adapter	<code>Arrays.asList</code> , <code>InputStreamReader</code> turning bytes into characters
Decorator	The whole of <code>java.io</code> , and <code>Collections.unmodifiableList</code>
Proxy	<code>java.lang.reflect.Proxy</code> , RMI stubs
Composite	<code>java.awt.Container</code> holding <code>Component</code> s
Strategy	<code>Comparator</code> , and every interface in <code>java.util.function</code>
Observer	Swing listeners, <code>PropertyChangeListener</code>
Publish-Subscribe, reactive streams	<code>Flow.Publisher</code> and <code>Flow.Subscriber</code>
Template Method	<code>AbstractList</code> , <code>AbstractMap</code> , <code>InputStream.read()</code>
Command	<code>Runnable</code> , <code>Callable</code>
Iterator	<code>Iterator</code> , <code>Iterable</code> , and the for-each loop
Flyweight	<code>Integer.valueOf</code> caching -128 to 127

Read this table as interpretation, not classification

These are the common readings rather than official labels, and several of these APIs show more than one pattern at once. `Collections.unmodifiableList` wraps like a decorator and restricts like a protection proxy. `Integer.valueOf` is a static factory and a flyweight cache. `Runnable` has a command's shape without a command's undo. `Collection.iterator()` has the shape of a factory method without the creator class the Gang of Four describe around it. `Flow` is filed above under its own publish-subscribe naming, though its publisher holds subscribers directly with no broker in between, so it is closer to Observer with backpressure added. In an interview, name the pattern and then say which part of it the API actually shows, because that is the answer that survives a follow-up.

20. Patterns in Spring

Pattern	In Spring
Singleton	The default bean scope. One instance per context, injected rather than looked up
Proxy	<code>@Transactional</code> , <code>@Cacheable</code> , <code>@Async</code> , <code>@PreAuthorize</code> , and all of AOP
Template Method, with callbacks	<code>JdbcTemplate</code> , <code>RestTemplate</code> , <code>TransactionTemplate</code>
Factory	<code>BeanFactory</code> , and <code>FactoryBean</code> for beans needing custom construction
Observer	<code>ApplicationEventPublisher</code> with <code>@EventListener</code>
Chain of Responsibility	The Spring Security filter chain, and servlet filters
Strategy	Injecting a <code>List</code> of one interface's implementations and picking between them
Adapter	<code>HandlerAdapter</code> , which lets the dispatcher call different controller styles
Front Controller	<code>DispatcherServlet</code> , the single entry point for every web request

Two rows need a note.

The template classes are the Template Method idea built with composition, not inheritance. The framework owns the fixed sequence, which for `JdbcTemplate` is acquiring a connection, executing, translating exceptions and releasing everything. The varying step arrives as a callback such as a `RowMapper` or a `StatementCallback`, rather than as a method overridden in a subclass. Saying that precisely is stronger than calling it textbook Template Method, because a sharp interviewer will point out that nothing is being subclassed.

The proxy row is the one that pays off most often, because it explains three separate behaviors an interviewer may ask about: why self-invocation loses the transaction, why a `final` method cannot be made transactional, and why AOP annotations do nothing on a `private` method.

21. Replacing a switch with Strategy

The starting point. Every new payment method edits this method, and the two others shaped like it:

```
public Money fee(String method, Money amount) {
    return switch (method) {
        case "card"    -> amount.percent(2.9);
        case "bank"    -> Money.ofCents(25);
        case "wallet"  -> amount.percent(1.5);
        default        -> Money.ZERO;    // silently wrong for an unknown method
    };
}
```

Each branch becomes a class behind one interface:

```
public interface FeePolicy {
    String method();
    Money feeFor(Money amount);
}

@Component
public class CardFee implements FeePolicy {
    public String method()        { return "card"; }
    public Money feeFor(Money amount) { return amount.percent(2.9); }
}

@Component
public class BankFee implements FeePolicy {
    public String method()        { return "bank"; }
    public Money feeFor(Money amount) { return Money.ofCents(25); }
}
```

Spring collects every implementation of the interface, so the lookup builds itself:

```

@Service
public class FeeCalculator {

    private final Map<String, FeePolicy> policies;

    public FeeCalculator(List<FeePolicy> all) {
        this.policies = all.stream()
            .collect(toMap(FeePolicy::method, policy -> policy));
    }

    public Money fee(String method, Money amount) {
        FeePolicy policy = policies.get(method);
        if (policy == null) {
            throw new IllegalArgumentException("unknown payment method: " + method);
        }
        return policy.feeFor(amount);
    }
}

```

Adding a wallet fee is now one new class annotated with `@Component`, and nothing above changes. Each policy is testable on its own, and the unknown method fails loudly instead of returning zero.

Worth saying before being asked: three lines of readable switch became three classes plus a registry. For a switch that has not changed in two years that is a bad trade. The pattern earns its place when new cases keep arriving, when each case is more than one line, or when the same switch appears in several places.

22. Interview questions

Each answer below is written to be said out loud as it stands. The last sentence always leads toward the next question rather than closing the topic.

"What is a design pattern?"

A named, reusable solution to a problem that keeps appearing in object-oriented design. It is not code we copy, it is a shape, described well enough that two engineers can say the name and picture the same arrangement. The catalogue people mean is the 1994 Gang of Four book, which describes twenty three of them. The real value is the shared vocabulary, because saying "the fee rules sit behind a Strategy" describes a whole design in a few words. The thing I would add is that a pattern is an answer, so the problem has to exist first, and applying one where it does not costs indirection for nothing.

"What are the three categories?"

Creational, structural and behavioral. Creational patterns are about how objects get made, so Singleton, Factory Method, Abstract Factory, Builder and Prototype. Structural patterns are about how objects are composed, so Adapter, Decorator, Facade, Proxy and Composite. Behavioral patterns are about how objects communicate and divide work, so Strategy, State, Observer, Template Method, Command and Chain of Responsibility. The ones I use most in ordinary Java work are Strategy, Adapter and Builder, and the one I am most careful with is Singleton.

"How would you implement a Singleton in Java, and would you?"

The version I would write is a single-element enum, which is what *Effective Java* recommends, because the language guarantees one instance, serialization works correctly and reflection cannot break it. If it has to be lazy I would use the holder idiom, where a private static nested class holds the instance, because class initialization is thread safe by specification and the class only loads on first use. Double-checked locking works too, but the `volatile` on the field is not optional, since without it another thread can see a reference to a partly constructed object. One precision worth adding is that the guarantee is per class loader rather than per JVM, so separate loaders give separate instances. On whether I would: rarely by hand, because the global access point hides a dependency instead of declaring it, and a test cannot substitute it. In Spring I get one instance per context anyway, but injected through the constructor.

"What is the difference between Adapter, Decorator and Proxy?"

Structurally they look the same, since all three wrap an object, and the difference is intent. An adapter changes the interface, so the caller can use something whose shape it otherwise could not, like wrapping a vendor SDK behind our own gateway interface. A decorator keeps the same interface and adds behavior, which is why decorators stack, and `java.io` is the example everyone has typed. A proxy also keeps the same interface but controls access to the real object, whether that is lazy loading, permission checks or a remote call. Spring's `@Transactional` is a proxy, and that is also why calling a transactional method from inside the same class starts no transaction.

"Strategy or State?"

They are the same shape and different intents, which is why the question gets asked. Both are an interface with several implementations and an object holding one of them. With Strategy the caller picks the algorithm, usually once, and the implementations are independent of each other, so a shipping cost calculator does not know any other shipping cost calculator exists. With State the object's whole behavior changes as it moves through a lifecycle, and the states often know their successors, so the paid state is the thing that returns the shipped state, though the transitions can sit in the context instead and it is still the same pattern. The practical signal is whether the implementations change during one object's lifetime. In Java I would often write the state version as an enum with behavior on each constant.

"Factory Method or Abstract Factory?"

Factory Method creates one product and lets a subclass decide which concrete class, so the choice is made by which subclass is in use. Abstract Factory creates a whole family of related products from one object, and its purpose is guaranteeing that the products match, so a Windows button never appears beside a Mac checkbox. The short version is one product versus one matching set. I would also separate both of those from a static factory method, which is just a named alternative to a constructor and is not a Gang of Four pattern at all, though it is the one I use most in Java.

"Strategy or Template Method?"

They solve the same problem from opposite directions. Template Method puts the fixed sequence in a base class and lets subclasses fill in steps, so it uses inheritance and the shared order lives in one place. Strategy puts each variation behind an interface and composes it in, so it uses composition and can be swapped at run time. I default to Strategy, because inheritance couples the subclass to the parent's internals and Java only allows one superclass. Template Method is still right when there really is one fixed sequence worth protecting, and then I make the template method final. Spring's `JdbcTemplate` is that idea, though built with callbacks rather than subclassing.

"Where do you see patterns in Spring?"

Almost everywhere, and the one worth leading with is Proxy, because it explains real behavior. Spring wraps beans in proxies to implement `@Transactional`, `@Cacheable` and `@Async`, which is why those annotations do nothing when the method is called from inside the same class, and why they cannot work on a final or private method. Beyond that, the default bean scope is Singleton but injected rather than looked up, `ApplicationEventPublisher` with `@EventListener` is Observer, the Spring Security filter chain is Chain of Responsibility, and `DispatcherServlet` is a Front Controller. `JdbcTemplate` is the Template Method idea implemented with callbacks instead of inheritance, which is worth saying precisely, because nothing is actually being subclassed.

"When would you not use a pattern?"

When the problem it answers has not appeared yet. The cost of a pattern is indirection, and indirection is only worth paying when something is actually going to change. A switch with three cases that has not been touched in two years does not need a Strategy, even though converting it would look more sophisticated. The failure mode I watch for is choosing a pattern because it is recognizable rather than because the code needs it, which produces four classes and an interface for something that was a clear ten-line method. My rule is to let the third change tell me where the seam belongs, and then name the pattern once it is already the shape the code wants.

23. Common mistakes

- Starting from the pattern instead of the problem, which buys indirection with no return.

- Writing a Singleton by hand inside a Spring application. Beans are already one instance per context, and the hand-rolled version loses the injection that makes tests possible.
- Forgetting `volatile` in double-checked locking. The code looks correct and fails rarely, under load, on a machine that is hard to reach.
- Claiming an object is immutable because its fields are `final`. A final field holding a mutable list is still a mutable object.
- Assuming a Spring annotation works on a self-invoked call. `@Transactional`, `@Cacheable` and `@Async` all go through a proxy, so an internal call skips them silently.
- Confusing Adapter, Decorator and Proxy because they have the same structure. The interviewer is asking about intent, so answer with intent.
- Putting `add` and `remove` on a Composite's shared interface, so leaves have to throw. That is a Liskov violation created by the pattern rather than solved by it.
- Putting cross-field validation in a builder's setter methods instead of in `build`, where the other values are not known yet.
- Using Observer for a flow that has to happen in order and cannot fail silently. Events decouple the caller, and they also make the sequence invisible in the code.
- Naming classes after patterns without cause, such as `OrderManagerFactoryStrategy`. If the name needs two pattern words, the design is usually the problem.

24. Quick reference

Pattern	Category	Intent in one line
Singleton	Creational	One instance per scope, one point of access
Factory Method	Creational	A subclass decides which concrete product to build
Abstract Factory	Creational	Create families of related products that must match
Builder	Creational	Build a complex object step by step, then freeze it
Prototype	Creational	Copy an existing object instead of building a new one
Adapter	Structural	Change one interface into the one a caller expects
Decorator	Structural	Wrap an object to add behavior, same interface
Facade	Structural	One simple entry point over a complicated subsystem
Proxy	Structural	Same interface, controls access to the real object
Composite	Structural	Treat a tree of objects like a single object
Flyweight	Structural	Share one instance across many uses to save memory
Strategy	Behavioral	Interchangeable algorithms behind one interface
State	Behavioral	Behavior changes as the object moves through its lifecycle

Pattern	Category	Intent in one line
Observer	Behavioral	Tell every dependent when the subject changes
Template Method	Behavioral	Fixed sequence in the base, steps filled by subclasses
Command	Behavioral	A request as an object, so it can be queued or undone
Chain of Responsibility	Behavioral	Pass a request along handlers until one deals with it
Iterator	Behavioral	Walk a collection without exposing how it is stored

Problem in the code	Pattern to consider	What it costs
A switch that grows with every new case	Strategy	One class per case, plus a registry to find them
A status field driving conditionals in many methods	State	One class or enum constant per state, and transitions to keep correct
A third-party class whose shape does not fit ours	Adapter	One more layer to write and keep in step with the vendor
Behavior needed on some calls but not others	Decorator	A stack of wrappers, and a longer path to read in a debugger
A constructor with six parameters, four optional	Builder	Roughly twice the code, unless a record or Lombok covers it
Callers must run three services in the right order	Facade	Another layer, which hides the subsystem from new readers
An expensive object built even when unused	Proxy, virtual	Indirection, and lazy failures far from the cause
A tree walked with instance of checks	Composite	A shared interface that has to suit leaves and groups alike
Several things must react to one event	Observer	The flow stops being readable in one file
Same sequence, one step differs each time	Template Method	Inheritance, and the fragile base class problem
A request needing queueing, logging or undo	Command	An object per action, and state to keep for undo
Requests filtered through ordered, optional steps	Chain of Responsibility	Order becomes configuration, so bugs move out of the code